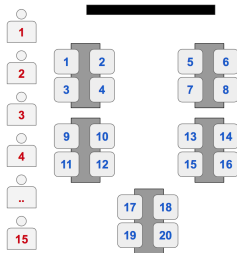


L04: Viewpoints and redundancy



Prof. Tias Guns and Dr. Hendrik 'Henk' Bierlee

KU LEUVEN

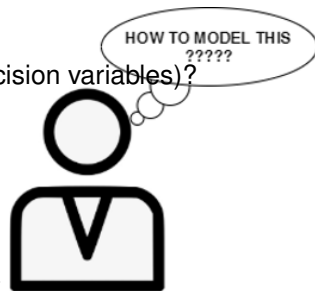
Partly based on slides from Pierre Flener, Barbara Smith and Dimos Tsouros.

Quick Recap

1. **Modelling**: express problem in terms of
 - ▶ parameters,
 - ▶ decision variables, with their domains
 - ▶ constraints, and
 - ▶ (optionally) objective.
2. **Solving**: solve using a state-of-the-art solver.

A Modelling Methodology: Overview

1. Make sure you understand the problem correctly:
 - ▶ What are the knowns (parameters) and unknowns (decision variables)?
 - ▶ What is the objective, if any?
2. Construct or select some small instance(s) of your problem to work with.
3. Model incrementally:
 - ▶ Add one constraint at a time and verify that it is correct.
 - ▶ Only add the objective function, if any, at the end.
4. Model first for correctness and clarity, and only then for efficiency:
 - ▶ Try to use the most specific type of global constraints which fits
 - ▶ Once you have a correct model, consider *redundant constraints* and symmetries.



In this lecture

We will become an *advanced* modeller!

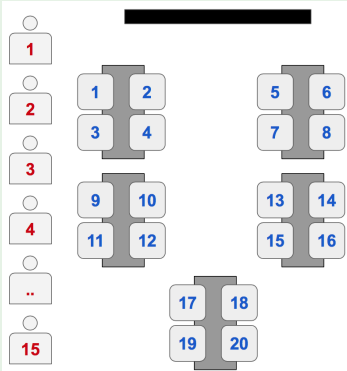
Importantly, the same problem can be represented by fundamentally different models.

- ▶ The **viewpoint** determines which unknowns to model as decision variables
- ▶ **Auxiliary** variables are useful, and sometimes required to model complex constraints
- ▶ **Redundant constraints** can improve the performance of the solver by adding knowledge to the solver which it cannot derive on its own
- ▶ **Channelling** constraints allow us to combine different viewpoints in the same model
- ▶ We have a look at some more general modelling guidelines

Outline

1. Viewpoints
2. Auxiliary variables
3. Redundant Constraints
4. Combining viewpoints: Redundant Variables & Channelling
5. Modelling guidelines

Example (Student Seating Problem (simplified))



`nStudents = 15`
`nChairs = 20 ≥ nStudents`
`nTables = 5`
`tables = [0..3, 4..7, 8..11, 12..15, 16..19]`

Given:

- ▶ `nStudents` students,
- ▶ `nChairs` chairs around `nTables` tables of four, and
- ▶ `tables[i]` as the set of chairs of table i .

Find a seating arrangement such that each table has either at least two of its chairs occupied, or else zero.

What are the **unknowns** of this problem? And what decision variables should we introduce?

Viewpoints

Definition (Law and Lee, 2002)

A **viewpoint** is a pair $\langle X, D \rangle$, where $X = \{x_1, \dots, x_n\}$ is a set of variables, and D is a set of domains. For each $x_i \in X$, the associated domain D_i is the set of possible values for x_i . It must be possible to ascribe a meaning to the variables and values of the CSP in terms of the problem P , and so to say what an assignment in the viewpoint $\langle X, D \rangle$ is intended to represent in terms of P .

Here, there are (at least) two **viewpoints**:

Viewpoint 1 Assign student i to chair j

Viewpoint 2 Assign chair i to student j

Which viewpoint will make the problem *easier to model*? And which viewpoint will make the problem *easier to solve*? Let us try both.

Viewpoint 1: assigning 15 students to 20 chairs

Let $\text{students}[i] = j$ denote that student i is assigned to chair j

Viewpoint 1: assigning 15 students to 20 chairs

Let $\text{students}[i] = j$ denote that student i is assigned to chair j

```
students = cp.intvar(0, n_chairs - 1, shape=n_students, name="students")  
model1 = cp.Model()
```

Viewpoint 1: assigning 15 students to 20 chairs

Let $\text{students}[i] = j$ denote that student i is assigned to chair j

```
students = cp.intvar(0, n_chairs - 1, shape=n_students, name="students")
model1 = cp.Model(
    cp.AllDifferent(students),
    cp.all(
```

Viewpoint 1: assigning 15 students to 20 chairs

Let $\text{students}[i] = j$ denote that student i is assigned to chair j

```
students = cp.intvar(0, n_chairs - 1, shape=n_students, name="students")
model1 = cp.Model(
    cp.AllDifferent(students),
    cp.all(
        (
            (cp.sum(students == chair for chair in table) == 0)
            | (cp.sum(students == chair for chair in table) >= 2)
        )
        for table in tables
    ),
)
```

Viewpoint 1: assigning 15 students to 20 chairs

Let $\text{students}[i] = j$ denote that student i is assigned to chair j

```
students = cp.intvar(0, n_chairs - 1, shape=n_students, name="students")
model1 = cp.Model(
    cp.AllDifferent(students),
    cp.all(
        (
            (cp.sum(students == chair for chair in table) == 0)
            | (cp.sum(students == chair for chair in table) >= 2)
        )
        for table in tables
    ),
)
```

The inner sums have $15 * 4 = 60$ terms.

Viewpoint 2: assigning 20 chairs to 15 students

Let $\text{chairs}[i] = j$ denote that chair i is assigned to student j .

Viewpoint 2: assigning 20 chairs to 15 students

Let $\text{chairs}[i] = j$ denote that chair i is assigned to student j . Use $\text{chairs}[i] = -1$ as a **dummy value** to represent that the chair is empty.

```
chairs = cp.intvar(-1, n_students - 1, shape=n_chairs, name="chairs")
model2 = cp.Model()
```

Viewpoint 2: assigning 20 chairs to 15 students

Let $\text{chairs}[i] = j$ denote that chair i is assigned to student j . Use $\text{chairs}[i] = -1$ as a **dummy value** to represent that the chair is empty.

```
chairs = cp.intvar(-1, n_students - 1, shape=n_chairs, name="chairs")
model2 = cp.Model(
    cp.AllDifferentExceptN(chairs, -1),
```

Viewpoint 2: assigning 20 chairs to 15 students

Let $\text{chairs}[i] = j$ denote that chair i is assigned to student j . Use $\text{chairs}[i] = -1$ as a **dummy value** to represent that the chair is empty.

```
chairs = cp.intvar(-1, n_students - 1, shape=n_chairs, name="chairs")
model2 = cp.Model(
    cp.AllDifferentExceptN(chairs, -1),
    cp.all(
        ((cp.sum(chairs[table] != -1) == 0) | (cp.sum(chairs[table] != -1) >= 2))
        for table in tables
    ),
)
```

Viewpoint 2: assigning 20 chairs to 15 students

Let $\text{chairs}[i] = j$ denote that chair i is assigned to student j . Use $\text{chairs}[i] = -1$ as a **dummy value** to represent that the chair is empty.

```
chairs = cp.intvar(-1, n_students - 1, shape=n_chairs, name="chairs")
model2 = cp.Model(
    cp.AllDifferentExceptN(chairs, -1),
    cp.all(
        ((cp.sum(chairs[table] != -1) == 0) | (cp.sum(chairs[table] != -1) >= 2))
        for table in tables
    ),
    cp.all(cp.any(chairs == i) for i in range(n_students)),
)
```

Viewpoint 2: assigning 20 chairs to 15 students

Let $\text{chairs}[i] = j$ denote that chair i is assigned to student j . Use $\text{chairs}[i] = -1$ as a **dummy value** to represent that the chair is empty.

```
chairs = cp.intvar(-1, n_students - 1, shape=n_chairs, name="chairs")
model2 = cp.Model(
    cp.AllDifferentExceptN(chairs, -1),
    cp.all(
        ((cp.sum(chairs[table] != -1) == 0) | (cp.sum(chairs[table] != -1) >= 2))
        for table in tables
    ),
    cp.all(cp.any(chairs == i) for i in range(n_students)),
)
```

Although additional constraints are needed, the inner sums have just 4 terms.

The choice of viewpoint will impact how easy certain constraints and objectives are to **model** or **solve**.

Fortunately, **easier to model** usually means **easier to solve** (but not always).

Consider the following when choosing your viewpoint:

- ▶ Expressiveness: some constraints or objectives are more easily expressed in one viewpoint than another, which impacts the **readability** of your model
- ▶ Performance: what is the strength of the constraints you are providing the solver?
 - ▶ Does your viewpoint allow for **strong** global constraints (e.g. ALLDIFFERENT over ALLDIFFERENTEXCEPTN)
 - ▶ What is the size of the search space?
 - ▶ Viewpoint 1: $\mathcal{O}(n_{\text{Chairs}}^{n_{\text{Students}}})$
 - ▶ Viewpoint 2: $\mathcal{O}((n_{\text{Students}} + 1)^{n_{\text{Chairs}}})$
 - ▶ $((15 + 1)^{20} \gg 20^{15})$
 - ▶ Does your viewpoint require you to nest complicated logical (e.g. disjunctions) or integer constraints (e.g. sub-expressions)?
 - ▶ In general, performance impact is **hard** to tell without running experiments!

Outline

1. Viewpoints
- 2. Auxiliary variables**
3. Redundant Constraints
4. Combining viewpoints: Redundant Variables & Channelling
5. Modelling guidelines

Auxiliary variables

Definition

Auxiliary variables are variables that are introduced in the model during the modeling process, and are not necessarily related to entities of our problem.

Auxiliary variables may be useful:

- ▶ Expressing constraints may be *easier* using auxiliary variables.
- ▶ Resulting model may be more *readable*.
- ▶ Propagation of constraints may be more *efficient*.

Auxiliary variables may be necessary:

- ▶ You may not be able to express certain constraints directly using the variables of the chosen *viewpoint*
- ▶ Depending on the used *solving technology/language*, auxiliary variables may need to be introduced to support nested constraints (using reification).

```

students = cp.intvar(0, n_chairs - 1, shape=n_students, name="students")
model1 = cp.Model(
    cp.AllDifferent(students),
    cp.all(
        (
            (cp.sum(students == chair for chair in table) == 0)
            | (cp.sum(students == chair for chair in table) >= 2)
        )
        for table in tables
    ),
)

```

An example where auxiliary variables are useful to avoid repeated expressions:

```

students = cp.intvar(0, n_chairs - 1, shape=n_students, name="students")
model1 = cp.Model(
    cp.AllDifferent(students),
    cp.all(
        (
            (cp.sum(students == chair for chair in table) == 0)
            | (cp.sum(students == chair for chair in table) >= 2)
        )
        for table in tables
    ),
)

```

An example where auxiliary variables are useful to avoid repeated expressions:

```

table_sums = [cp.sum(students == chair for chair in table) for table in tables]
model3 = cp.Model(
    cp.AllDifferent(students),
    cp.all(((t == 0) | (t >= 2)) for t in table_sums),
)
model3.add(cp.all(t <= 4 for t in table_sums))

```

The solver can benefit if the same expression is not introduced twice (although Common Subexpression Elimination (CSE) is in place to do this automatically).

Example (Car Sequencing; Dincbas, Simonis and van Hentenryck, 1988)

Find a **sequence** in which to produce 10 cars according to the following table:

Classes	1	2	3	4	5	6	Cap.
No. of cars	1	1	2	2	2	2	
Option 1	✓	✓			✓	✓	1/2
Option 2			✓	✓		✓	2/3
Option 3	✓				✓		1/3
Option 4	✓	✓		✓			2/5
Option 5			✓				1/5

- ▶ The **class** of each car determines how many cars to produce, and with which options. E.g. for class 1, we should produce 1 car with options 1, 3, and 4.
 - ▶ E.g. a sequence of car classes: [1, 2, 3, 3, 4, 4, 5, 5, 6, 6]
- ▶ The stations that install the options have limited **capacities**. E.g. for option 1, we can only produce 1 car in every sub-sequence of 2 cars. Producing a car of class 1 can thus only be directly followed by a car of class 3 or 4.

Auxiliary variables - Car Sequencing

- ▶ Viewpoint:

- ▶ A variable for each position in the sequence,
 $s_1, s_2, \dots, s_{10} \in S$
- ▶ Let $s_i = j$ denote that the i -th car is of class j

- ▶ Constraints:

- ▶ Each class occurs the correct number of times
 $\text{COUNT}(s, k) = d_k \quad \forall k \in \text{Classes}$
with d_k being the production demand for class k . This posts e.g.
 $\text{COUNT}(s, 3) = 2$ to ensure we build 2 cars of class 3.

Or better: *GlobalCardinalityCount*($S, \text{Classes}, D$)
with D being the list of production demands for each class.

- ▶ How to model the option given the capacities? This is (almost) impossible given the current viewpoint.



Auxiliary variables - Car Sequencing

How can we model the constraints for the option capacities?

- ▶ Introduce **auxiliary** variables o_{ij} :

- ▶ $o_{ij} = 1$ iff the car in the i -th slot in the sequence requires option j



Auxiliary variables - Car Sequencing

How can we model the constraints for the option capacities?

- ▶ Introduce **auxiliary** variables o_{ij} :
 - ▶ $o_{ij} = 1$ iff the car in the i -th slot in the sequence requires option j
- ▶ Capacity constraint for e.g. option 2 with capacity of 2/3
 - ▶ Constrain a sliding window of size 3
 - ▶ $o_{i,2} + o_{i+1,2} + o_{i+2,2} \leq 2$ for $1 \leq i \leq 8$



Auxiliary variables - Car Sequencing

How can we model the constraints for the option capacities?

- ▶ Introduce **auxiliary** variables o_{ij} :

- ▶ $o_{ij} = 1$ iff the car in the i -th slot in the sequence requires option j

- ▶ Capacity constraint for e.g. option 2 with capacity of 2/3

- ▶ Constrain a sliding window of size 3

- ▶ $o_{i,2} + o_{i+1,2} + o_{i+2,2} \leq 2$ for $1 \leq i \leq 8$

- ▶ Relate the auxiliary variables to the s_i variables:

- ▶ Matrix $\lambda_{kj} = 1$ if car class k requires option j

- ▶ If $s_i = k$, then the options of class k are required in slot i

- ▶ $o_{ij} = \lambda_{s_i j}$ $1 \leq i \leq 10, 1 \leq j \leq 5$ **Note: this uses in practice the Element global function. A variable (s_i) is used as index into the rows of λ .**



Auxiliary variables – Car Sequencing – CPMpy

Example (CPMpy model for Car Sequencing (indexing offset 0))

```
# Sequence of different car types
sequence = cp.intvar(0, n_classes - 1, shape=n_cars, name="sequence")
# Auxiliary variables for options
option = cp.boolvar(shape=(n_cars, n_options), name="option")

# Each car class occurs the correct number of times
model.add(cp.GlobalCardinalityCount(sequence, range(n_classes), demand))

# connect auxiliary variables with decision variables
for s in range(n_cars):
    model.add([option[s, o] == requires[sequence[s], o] for o in range(
n_options)])

# Option capacities are respected
for o in range(n_options):
    for s in range(n_cars - per_slots[o] + 1):
        slotrange = range(s, s + per_slots[o])
        model.add(cp.sum(option[slotrange, o]) <= at_most[o])
```

Outline

1. Viewpoints
2. Auxiliary variables
- 3. Redundant Constraints**
4. Combining viewpoints: Redundant Variables & Channelling
5. Modelling guidelines

Redundant constraints

We have chosen a viewpoint, and formulated the constraints to express our problem in a **correct** and **readable** model. But, can we do something to make the solving more **efficient**?

During the search process, the solver will explore several infeasible parts of the search tree when it cannot propagate constraints. Can we avoid that?

Definition

Redundant constraints, also called *implied* constraints, are constraints which are entailed by the constraints defining the problem. They do not change the set of solutions, and hence are logically redundant.

Although **redundant** constraints are logically redundant, and do not change the set of solutions, they can be used to **reduce the search effort of the solving process**.

Redundant constraints

Example

Because of the viewpoint, it is impossible to assign more than four students to the same table. However, we can add the following redundant constraint:

```
model3.add(cp.all(t <= 4 for t in table_sums))
```

Redundant constraints are often added to the model to reduce the search effort, by **cutting infeasible parts** of the search tree early. So the solver does not need to explore possible assignments in these subproblems.

- ▶ at some point during search, we will have a partial assignment that cannot lead to a solution
- ▶ the redundant constraint can detect inconsistency of the partial assignment
 - ▶ the redundant constraint forbids it, but **does not change the set of solutions**
- ▶ while without the redundant constraint, the search would continue deeper

Redundant constraints – Car Sequencing

- ▶ Existing constraints only say that the option capacities cannot be *exceeded*
 - ▶ but we can't stay too far below capacity either
- ▶ Suppose there are 30 cars, and 12 require option 1
- ▶ Reminder: Option 1 capacity is 1 car in 2
- ▶ At least one car in slots 1 to 8 of the production sequence must have option 1
 - ▶ otherwise 12 of cars 9 to 30 will require option 1, i.e. too many
- ▶ Cars 1 to 10 must include at least two option 1 cars, ..., and cars 1 to 28 must include at least 11 option 1 cars
- ▶ These are *redundant constraints* that can be used to speed up search in car sequencing



Example (Magic Series of length n)

The element at index i in $I = 0 \dots (n-1)$ is the number of occurrences of i . A solution for $n = 4$: `Magic = [1, 2, 1, 0]` (0 occurs once, 1 occurs twice, ...)

Decision variables: $\text{Magic} =$

0	1	$\dots$	$n-1$
$\in I$	$\in I$	$\dots$	$\in I$

Problem Constraint:

$$\forall i \in I, \quad \text{Magic}[i] = \sum_{j \in I} (\text{Magic}[j] = i)$$

or, logically equivalently but better:

$$\forall i \in I, \quad \text{CountEq}(\text{Magic}, i, \text{Magic}[i])$$

or, logically equivalently and even better:

$$\text{GlobalCardinalityCount}(\text{Magic}, \text{array1d}(I, [i \mid i \in I]), \text{Magic})$$

Example (Magic Series of length n)

The element at index i in $I = 0 \dots (n-1)$ is the number of occurrences of i . A solution for $n = 4$: `Magic = [1, 2, 1, 0]` (0 occurs once, 1 occurs twice, ...)

Decision variables: `Magic` =

0	1	$\dots$	$n-1$
$\in I$	$\in I$	$\dots$	$\in I$

Problem Constraint:

```
[Magic[i] == cp.sum(Magic == i) for i in range(len(Magic))]
```

or, logically equivalently but better:

```
[Magic[i] == cp.Count(Magic, i) for i in range(len(Magic))]
```

or, logically equivalently and even better:

```
cp.GlobalCardinalityCount(Magic, range(len(Magic)), Magic)
```

Redundant constraints

We have modeled the problem correctly. Example solutions:

$\text{Magic} = [1, 2, 1, 0], [2, 0, 2, 0], \dots$ What redundant constraints can we add?

Redundant Constraints: $\sum \text{Magic} =$

Redundant constraints

We have modeled the problem correctly. Example solutions:

$\text{Magic} = [1, 2, 1, 0], [2, 0, 2, 0], \dots$ What redundant constraints can we add?

Redundant Constraints: $\sum \text{Magic} = n$ and $\sum_{i \in I} (i * \text{Magic}[i]) =$

Redundant constraints

We have modeled the problem correctly. Example solutions:

$\text{Magic} = [1, 2, 1, 0], [2, 0, 2, 0], \dots$ What redundant constraints can we add?

Redundant Constraints: $\sum \text{Magic} = n$ and $\sum_{i \in I} (i * \text{Magic}[i]) = n$

The first holds because we sum up counts of occurrences of each number, and there are n numbers.

The second is more tricky. Because $\text{Magic}[i] = j$ means there are j occurrences of value i , placed at some indices $k_1, \dots, k_j$, which in turn ensures $\text{Magic}[k_1], \dots, \text{Magic}[k_j]$ numbers are placed. However, now we are counting occurrences again ($\text{Magic}[k_1]$ number of k_1 's, etc.), which always should add up to n .

Depending on the formulation above of the problem constraint, the redundant constraints can accelerate a CP solver up to 100 times for $n = 150$.

Redundant Constraints

Be careful though: not all redundant constraints accelerate the solving.

- ▶ The assignments forbidden by a redundant constraint may never actually arise during search
 - ▶ This means more propagation time and no benefit
- ▶ E.g. in car sequencing,
 - ▶ at least one of the first 8 cars must require option 1
($o_{1,1} + o_{2,1} + o_{3,1} + \dots + o_{8,1} \geq 1$)
 - ▶ in general, *any* 8 consecutive cars must have one option 1 car
 - ▶ but this redundant constraint is quite weak, and is thus less likely to pay off
- ▶ If we find a *class* of redundant constraints, maybe only some are useful
 - ▶ adding a lot of constraints that don't reduce search will increase the run-time



Outline

1. Viewpoints
2. Auxiliary variables
3. Redundant Constraints
- 4. Combining viewpoints: Redundant Variables & Channelling**
5. Modelling guidelines

Combining viewpoints

Adding **auxiliary** variables may be important to model your problem (efficiently), while adding **redundant constraints** can speed up the search process.

Can we do something more to get a more efficient model??

We may have spotted different viewpoints for our problem ...

And one is not *clearly* better than the other:

Different perspectives often allow different expression of the constraints and different redundant constraints

Why choose one of them? Can use both!

Combining viewpoints

We have chosen our main viewpoint for the problem,

- ▶ but we also introduce a second viewpoint with the hope that it will help in propagation during search.

This means we will use **redundant variables**, that do not capture information not already captured in the model.

Definition

A **redundant decision variable** denotes information already denoted by other variables: **mutual redundancy** (same information) vs **non-mutual redundancy**.

The constraints of the second viewpoint will be used on these variables.

Channelling

Including the variables and constraints of both viewpoints may allow the solver to infer different things from each one, and reduce the search space.

But we have to make sure that our 2 sets of variables solve the same problem!

We need to channel the 2 viewpoints, linking the variables i.e. make sure that assignments (and domain pruning) on one viewpoint are propagated to the other.

Definition

A **channelling constraint** fixes the value of 1 set of decision variables when the values of the decision variables they are redundant with from the second set are fixed. Channelling can be **1-way** (one viewpoint determines the other) or **2-way** (assignments and domain pruning propagate in both directions).

Channelling

Example (Golomb Rulers)

- ▶ A Golomb ruler with m marks consists of:
 - ▶ a set of m integers $x_1, x_2, \dots, x_m$ (marks on the ruler)
 - ▶ all differences $x_j - x_i$ (for $i < j$) are all different
 - ▶ Objective: find ruler with minimum length $\max_i x_i$
- ▶ First viewpoint: ordered marks
 $0 = x_1 < x_2 < \dots < x_m$
 - ▶ $\text{ALLDIFFERENT}(\{x_j - x_i : 1 \leq i < j \leq m\})$
 - ▶ $x_1 < x_2 < \dots < x_m$
(marks are unordered in the problem; ordering is a modelling choice — also symmetry breaking, next lecture)

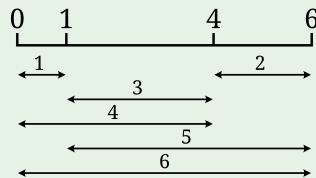


Figure: A Golomb ruler for $m = 4$ where $x = [0, 1, 4, 6]$

What about the Student Seating problem?

Try it in the exercise session

Outline

1. Viewpoints
2. Auxiliary variables
3. Redundant Constraints
4. Combining viewpoints: Redundant Variables & Channelling
- 5. Modelling guidelines**

From a Problem to a Model

What is a good model for a constraint problem?

- ▶ A model that **correctly** represents the problem
- ▶ A model that is **easy** to understand and maintain
- ▶ A model that is solved **efficiently**, that is:
 - ▶ **short** solving time to find one, all, or best solution(s)
 - ▶ **good** solution within a limited amount of time
 - ▶ **small** search space (under systematic search)

Food for thought: what is **correct**, **easy**, **short**, **good**, ... ?

Modelling Issues

Modelling is still more a craft than a science:

- ▶ Choice of viewpoint:
 - ▶ which are the decision variables and their domains?
 - ▶ will we use different viewpoints for different constraints?
- ▶ Choice of the constraint formulations
- ▶ Choice of solver and solver configuration (later lectures)

Make a model correct before making it efficient!

Tip: declare the Decision Variables with Tight Domains

Tight domains for decision variables might accelerate the solving.

Think about the bounds of your integer variables, avoid lazily putting a generic large constant.

Example (Think about tight domains)

If you have decision variable $x \in \{1, \dots, 20\}$ and $y \in \{5, \dots, 10\}$, then if you want to create an explicit decision variable with $z = \text{MAX}(x, y)$, think it through:

$$z \in \{5, \dots, 20\}$$

$$z = \text{MAX}(x, y)$$

minimize z

If your modeling library allows it, even better is to use nested expressions; the library will decompose and create an auxiliary variable only if the solver needs it:

minimize $\text{MAX}(x, y)$

Tip: beware of Division Constraints

The use of *division* and *modulo* on decision variables often makes the solving slow.

Reformulate where possible.

Example (Avoid unnecessary divisions)

If you want to minimize the (integer) average value of an array of decision variables X :

$$\text{minimize } \sum X / \text{len}(X)$$

then realizing that $\text{len}(X)$ is a constant, you can reformulate the objective as:

$$\text{minimize } \sum X$$

given that you post-process the objective value by dividing it by $\text{len}(X)$.

Tip: avoid floating point constants in pure integer models

Example

The following constraint is inefficient or unsupported for many solvers:

$$\sum X \geq (2/3) \cdot b$$

Instead, bring the denominator to the other side:

$$3 \cdot \sum X \geq 2 \cdot b$$

Summary

- ▶ Find the different viewpoints that can be used to model the problem
 - ▶ Is one of them clearly better at formulating the constraints?
 - ▶ Do they give better propagation on a part of the problem?
 - ▶ Should you combine viewpoints?
- ▶ Express the problem concisely: global constraints and functions
- ▶ Possibly use redundant constraints to speed up the search
- ▶ Use few decision variables, and declare tight domains
 - ▶ Auxiliary variables may be needed to formulate some of the constraints
- ▶ Beware of division constraints and floating-point values

Above all: have fun and try different things!